# Experience Report: Simulation Tool Capabilities

CATEC

July 9, 2026

**Abstract**

Practitioner findings for GMAT, Orekit, Basilisk, FreeFlyer, and NASA 42, organized by the same capability groups as the state-of-the-art matrix. Intended for engineers selecting a platform for a new mission.

## Contents

**1 Benchmark concepts tested (deduplicated summary) 2**

**2 General 3**
2.1 Open source . . . 3
2.2 User interface and automation . . . 3
2.3 MATLAB integration . . . 3

**3 Core astrodynamics / mission analysis 3**
3.1 Orbit propagation . . . 3
3.2 Maneuver planning / targeting . . . 4
3.3 Multi-spacecraft / constellation modeling . . . 4
3.4 Interplanetary / multi-celestial-body missions . . . 4

**4 Advanced mission analysis 5**
4.1 Trajectory / orbit optimization . . . 5
4.2 Orbit determination . . . 5
4.3 Monte Carlo analysis . . . 5
4.4 Collision / conjunction analysis . . . 5
4.5 Contact / coverage analysis . . . 6

**5 GNC / simulation 7**
5.1 Spacecraft dynamics & GNC simulation . . . 7
5.2 Flight software development . . . 7
5.3 Real-time wall-clock / HIL simulation . . . 7
5.4 Live visualization during simulation . . . 8

**6 Mission operations 9**
6.1 Mission operations support . . . 9

**7 Software architecture 9**
7.1 Plugin / extension architecture . . . 9

**8 Summary table 12**

**9 Recommended tools by capability 14**

**10 Selection guide 15**

# 1 General

## 1.1 Open source

GMAT, Orekit, Basilisk, and NASA 42 are open source and suitable for reproducible internal workflows. FreeFlyer requires a commercial license.

## 1.2 User interface and automation

**GMAT and FreeFlyer** are **desktop applications**. Their GUI is not limited to upfront layout: it covers the analyst workflow end to end—resources (spacecraft, propagators, force models, ground stations, sensors, formations), mission sequence (propagate, maneuvers, targeting, differential correction, reports, plots), and stepped or live 3D views. Short studies can be built entirely in the GUI; multi-spacecraft setups with different force models per vehicle, long batch campaigns, and repeatable benchmark exports in this evaluation were maintained as scripts rather than rebuilt by hand each time.
**GMAT** is driven by **GMAT Script** (`.script`; MATLAB-like syntax, not MATLAB) and documents a **Python interface** for batch automation.
**FreeFlyer** uses **FreeForm** (`.ff`, usually embedded in a Mission Plan) and a **Runtime API** (Python, MATLAB, Java, C/C++); complex logic in this evaluation lived in FreeForm blocks inside the plan.
**Orekit** is a **Java** library; Python access uses the community `orekit-jpype` wrapper. There is no Orekit desktop console—propagation, maneuvers, estimation, and coverage are assembled in application code.
**Basilisk** missions are wired in **Python** scripts that bind **C/C++** simulation modules through SWIG; extending the framework with new dynamics or sensor blocks is a C/C++ task, but analysts do not author full missions in C/C++.
**NASA 42** is driven by text input decks and an optional OpenGL viewer; it has no Python, MATLAB, or vendor script API.

## 1.3 MATLAB integration

For a hybrid workflow—one tool for propagation or targeting, the rest in MATLAB—only **GMAT** and **FreeFlyer** document a host API callable from MATLAB. GMAT's API (R2020a onward) lets MATLAB load a `.script`, change parameters, run the mission, and read runtime objects or report files back into MATLAB arrays. FreeFlyer's Runtime API lists MATLAB alongside Python, Java, and C/C++ for the same pattern: execute a plan and pull outputs into the host environment. **Orekit**, **Basilisk**, and **NASA 42** have no official MATLAB API; coupling **Basilisk** or **NASA 42** to MATLAB means custom glue (Python bridges or file exchange on exported ephemeris and CSV). While **Orekit** also lacks an explicit MATLAB API, MATLAB natively hosts a Java virtual machine, so analysts can add the Orekit `.jar` to `javaclasspath` and call Orekit Java classes directly from MATLAB scripts without an external wrapper.

# 2 Core astrodynamics / mission analysis

## 2.1 Orbit propagation

All evaluated tools propagate Earth orbits credibly. A full one-year Earth propagation with drag, SRP, and third-body perturbations is routinely executed offline on a workstation in an average of about ten minutes.
Native astrodynamics catalogs differ; all tools cover two-body motion, J2, atmospheric drag, SRP, and Sun/Moon third-body in official documentation or examples, but not every perturbation class appears everywhere.

**FreeFlyer** ships Earth spherical harmonics up to 360×360 (EGM96) and solid tides; built-in atmospheres include **NRLMSISE-00**, earlier **MSIS-family** models (MSIS-2000/90/86), Jacchia-Roberts, and Harris-Priester.
**GMAT** takes harmonic degree and order from the chosen potential file, includes **NRLMSISE-00** drag, relativistic correction, and scriptable solid, pole, and ocean tides, plus polyhedral gravity.
**Orekit** documents the broadest perturbation set in library form: high-degree harmonics from standard gravity files, NRLMSISE-00 and several Jacchia/DTM atmospheres, solid and ocean tides, general relativity, Earth albedo/infrared, and shaped SRP—all assembled in application code.
**Basilisk** propagates through SPICE ephemerides with user-selected harmonic degree (from coefficient files), polyhedral gravity, **MsisAtmosphere** (NRLMSISE-00), and modular drag/SRP effectors wired to spacecraft dynamics; it does not document ocean tides, general relativity, or albedo forcing at the Orekit level.
**NASA 42** is built for multibody attitude–orbit dynamics with planetary ephemerides, not for tuning high-order Earth gravity fields in a mission-analysis workflow.

### 2.2 Maneuver planning / targeting

GMAT, FreeFlyer, and Orekit all support closing orbit goals with impulsive or finite burns. GMAT and FreeFlyer provide `Target` segments in the mission sequence for apogee/perigee maneuvers, plane changes, rendezvous-style goal sets, and reset loops. Orekit exposes the same class of problem through maneuver objects, event detectors, and Lambert solvers wired in application code. Basilisk can apply impulsive velocity updates or Lambert guidance modules, but has no native mission targeter—operational element goals are closed in a custom script loop around the propagator. NASA 42 is not used for mission-analysis burn targeting.

### 2.3 Multi-spacecraft / constellation modeling

FreeFlyer is the strongest option for constellation design: it supports Walker layouts, named constellation presets, formation propagation, coverage links, and one integrated 3D view. GMAT supports multi-spacecraft formations and formation-flying visualization, but constellation pattern generation is more manual. Orekit provides Walker slot builders and accurate propagation per vehicle, but visualization and merged analysis are application code. Basilisk supports Walker-style multi-satellite simulations and relative-navigation architectures, but constellation design and coverage analysis are custom Python work. NASA 42 can simulate many spacecraft with formation-relative states, but is not a constellation-design or coverage-analysis tool.

### 2.4 Interplanetary / multi-celestial-body missions

GMAT, FreeFlyer, Orekit, and Basilisk all support multi-celestial-body propagation through planetary ephemerides and third-body dynamics. GMAT and FreeFlyer are strongest when the project needs trajectory design and refinement for lunar, planetary, flyby, or low-thrust transfers. Orekit covers the same multi-body analysis foundation, including CR3BP/libration-point workflows, when the mission software owns the design logic. Basilisk is strong for SPICE-defined planetary flyby simulation, visualization, and attitude/GNC behavior along the trajectory, but not for choosing the transfer or optimizing the flyby. NASA 42 supports planetary, escape, Lagrange-point, and deep-space dynamics, but is not a trajectory-design tool.

# 3 Advanced mission analysis

## 3.1 Trajectory / orbit optimization

GMAT and FreeFlyer are the strongest choices when trajectory refinement is a primary project requirement: both support multi-variable targeting, transfer design, station-keeping style goals, and finite- or low-thrust optimization patterns. GMAT exposes solver progress through its differential-correction and optimizer workflow; FreeFlyer has rich mission-tier examples for electric propulsion, rendezvous, lunar assists, and Earth–planet transfers. Orekit provides the propagation, maneuver, Lambert, and event-detection foundation, but the optimizer is supplied by the surrounding application. Basilisk is useful for testing a candidate trajectory with spacecraft dynamics and FSW behavior, but trajectory optimization itself is external Python work. NASA 42 is not a mission trajectory optimizer.

## 3.2 Orbit determination

GMAT and FreeFlyer are the strongest ready-to-use ground orbit-determination workflows.
**GMAT:** navigation scripts cover DSN range/Doppler simulation and batch estimation, simultaneous multi-spacecraft estimation, initial covariance, thrust scale and angle estimation, TDRS/user tracking, range/range-rate, angle data, drag/SRP scale-factor estimation, and residual/covariance reporting.
**FreeFlyer:** larger mission-plan library around batch least-squares, Kalman, unscented Kalman, and square-root information filters with ground, spacecraft, GNSS/Galileo, TDRS, lunar, custom, and maneuver-estimation measurements.
**Orekit:** strongest embedded OD library; batch least-squares, Kalman/unscented Kalman estimators, measurement models, modifiers, and covariance handling are available in Java/Python application code.
**Basilisk:** useful for onboard navigation, optical navigation, small-body navigation, and estimator/FSW simulation, not as a ground OD processing desk.
**NASA 42:** not an orbit-determination tool.

## 3.3 Monte Carlo analysis

**FreeFlyer and Basilisk** are the clearest choices for ready Monte Carlo-style campaigns. **FreeFlyer** includes mission-plan examples for general Monte Carlo runs and probability-of-collision sampling. **Basilisk** includes Monte Carlo controllers, rerun/visualization utilities, and attitude/opnav dispersion examples. **GMAT** can be driven through scripts or the API for repeated runs, but no dedicated Monte Carlo campaign example appears in the sample set reviewed. **Orekit** uses user loops or Taylor-field propagation rather than a campaign manager. **NASA 42** supports noise/random-walk parameters inside spacecraft simulations, but Monte Carlo campaigns must be scripted externally.

## 3.4 Collision / conjunction analysis

**FreeFlyer** is the strongest ready-to-use option for close-approach work: its examples include probability-of-collision and Monte Carlo probability-of-collision workflows. **Orekit** is the strongest embedded SSA library: it provides CCSDS CDM support, state covariance handling, and multiple probability-of-collision methods for short-term encounters. **GMAT** is useful as a high-fidelity propagation and OD engine for scripted conjunction studies and Monte Carlo Pc estimation, but it is not a ready conjunction-screening desk. **Basilisk** can simulate relative motion and collision scenarios, but Pc calculation and operational screening are external analysis. **NASA 42** is not suited to operational conjunction screening.

### 3.5 Contact / coverage analysis

**FreeFlyer** is the strongest ready-to-use coverage tool: it includes spacecraft contact/access, constellation contact, chain contact, regions, point groups, sensors, link-style examples, hurricane coverage, and coverage-time optimization. **GMAT** is strong for ground-station contact, station masks, light-time options, and eclipse reporting, but not for rich coverage products. **Orekit** is strong as an embedded access engine through elevation, eclipse, field-of-view, and event detectors; reports and visualization are application work. **Basilisk** supports point access, eclipse, spacecraft-location, ground-location imaging, and strip imaging, but coverage grids, revisit metrics, and polygon reports are custom Python work. **NASA 42** has sensors and geometry in the dynamics simulation, but not a coverage-analysis/reporting workflow.

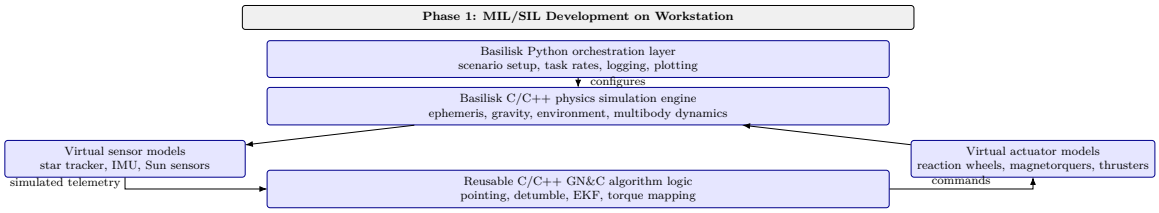
# 4 GNC / simulation

## 4.1 Spacecraft dynamics & GNC simulation

**Basilisk and NASA 42** are the strongest choices for spacecraft dynamics and GNC simulation: both support coupled orbit/attitude dynamics, sensors, actuators, reaction-wheel-based attitude control, thrusters, and closed-loop control studies. **Basilisk** is more convenient for scripted campaigns, Monte Carlo runs, and Vizard-based visualization; **NASA 42** is especially useful for 6-DOF dynamics, flexible bodies, and input-deck control-law studies. **GMAT and Orekit** provide attitude models/providers for mission analysis, not native ADCS actuator simulations. **FreeFlyer** supports attitude states, attitude propagation, moments of inertia, and environmental torque quantities, but not a native reaction-wheel ADCS control-loop workflow.

## 4.2 Flight software development

**Basilisk** is the strongest tool for MIL/SIL-style FSW development when the goal is a large library of reusable C/C++ GNC modules (`fswAlgorithms`) orchestrated from Python: processes, tasks, and message buses mirror flight-software structure, so algorithm logic exercised in simulation can carry forward to later onboard integration. It does not generate turn-key flight firmware; OBC cross-compilation, hardware drivers, scheduling, safety logic, and qualification remain project work.
**NASA 42** supports closed-loop ADCS/GNC through spacecraft decks (`PROTOTYPE_FSW` or custom FSW identifiers), `Inp_Cmd.txt` pointing commands, C extensions in `42fsw.c`, and ACS IPC sockets for external sensor/actuator exchange. This is C/C++ GNC integration without a full onboard software stack.
For **onboard cFS flight software** (RTOS apps, CCSDS telemetry, ground commanding), **NOS3** is the practical wrapper around the same NASA 42 input decks: 42 remains the multi-body dynamics and environment engine, while NOS Engine routes sensor and actuator traffic between 42 (`Inp_IPC.txt`), hardware simulators, and cross-compiled **cFS** applications. Spacecraft, orbit, actuator, and sensor definitions prepared for stand-alone 42 (for example a GMAT BSP converted to a `SPLINE` reference orbit) transfer to NOS3 with configuration changes, not a separate physics model. Basilisk still wins on Python-native module iteration; NOS3 wins when the deliverable is flight-like C/C++ FSW talking to simulated avionics.
**GMAT, FreeFlyer, and Orekit** do not target onboard software iteration.

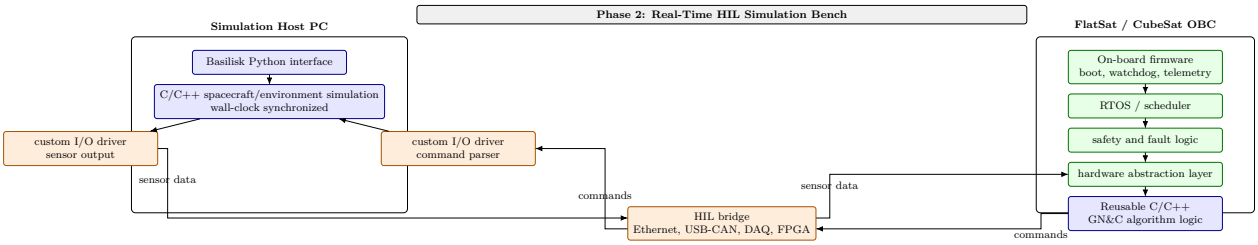



**Basilisk MIL/SIL software architecture.** Reusable C/C++ GN&C logic is exercised inside the workstation simulation.

## 4.3 Real-time wall-clock / HIL simulation

**Basilisk** can support HIL-style experiments through wall-clock synchronization, real-time execution control, and custom interfaces to external hardware or flight software. In this setup Basilisk simulates the spacecraft, environment, sensors, and actuator response, while the hardware under test runs the controller or flight software. It is not a turnkey HIL platform: deterministic I/O, processor-in-the-loop execution, communication bridges, and timing qualification must be engineered for the target bench.
Stand-alone **NASA 42** real-time OpenGL execution is interactive dynamics on the simulator host—useful for operator review, but not processor-in-the-loop with an external OBC.
**NOS3** provides the HIL path for the same 42-based plant model: NOS Engine synchronizes 42, software-only hardware simulators, and cFS instances on wall-clock ticks (`Inp_NOS3.txt`), with IPC

between 42 and the simulator stack (`Inp_IPC.txt`). A flatsat or OBC can replace the Linux-hosted cFS binary once bus interfaces are mapped. This is closer to an operational FSW test bench than Basilisk-in-the-loop, but setup and integration cost are higher. For CATEC, a practical split is Basilisk for fast algorithm MIL/SIL and custom HIL glue; NOS3 when the project already targets cFS and needs ground plus simulated avionics in the loop.
**GMAT, FreeFlyer, and Orekit** are not native ADCS HIL platforms.




**Basilisk HIL workflow.** Basilisk simulates the spacecraft/environment while the target hardware runs the controller.

## 4.4 Live visualization during simulation

- **True online / second-by-second: Basilisk Vizard** supports live 3D simulation viewing during execution through Direct Communication over TCP, with optional saved playback. **NASA 42** also provides an optional live OpenGL front end with a configured graphics output interval, so it can show the simulation as it runs when graphics performance is adequate. Both are live 3D viewers, but they do not provide native live XY/debug plots like GMAT and FreeFlyer.
- **Semi-live / runtime batched: GMAT** OpenFrames/OrbitView and **FreeFlyer** Map/3D views update during a running mission sequence, but their refresh is tied to the script/mission execution and configured update intervals. They are useful for following propagation live, and their XY/debug plots follow the same semi-live update model; heavy solvers, Monte Carlo runs, or iterative batches may update in chunks rather than stream continuously.
- **Offline / post-processed: Orekit** has no native live 3D engine. OreCzml/Cesium is best treated as an offline export and replay path; true streaming Cesium requires custom server/client work and can lag for complex scenes.

# 5 Mission operations

## 5.1 Mission operations support

This capability means support for post-launch *ground* flight-dynamics work in a CubeSat operations center—not onboard software and not a full telemetry/command console.
A typical workflow starts when a ground station records tracking data during a pass (Doppler, range, or range-rate). That data is delivered to an analyst workstation and loaded into a flight-dynamics tool, which runs orbit determination on the past measurements, propagates the recovered state forward faster than real time, and produces operations products: updated ephemeris, predicted contact windows, eclipse intervals, and, when needed, a maneuver plan (for example a debris-avoidance burn with burn time, direction, and $\Delta v$). Operators uplink that maneuver product through the radio link; **GMAT** and **FreeFlyer** do not execute it on the spacecraft.
**GMAT and FreeFlyer** are the native choices for this first-step ground workflow: OD filters, propagation, event and access prediction, targeting/differential correction, reporting, and repeatable script or API runs without rebuilding the pipeline in custom code.
**Orekit** provides the same underlying flight-dynamics functions as an embedded Java/Python library for custom operations software, not as a ready analyst console.
**Basilisk** is not a routine mission-operations desk, but it has an important second-step role once a maneuver or ephemeris exists. Teams load the planned burn or trajectory from **GMAT**/**FreeFlyer** (or from exported products) into a high-fidelity simulation and verify that reaction wheels, thrusters, closed-loop FSW, and attitude constraints can execute it—for example slew timing, wheel desaturation, sun exclusion, and telemetry replay through onboard algorithm modules. **Basilisk** *can* in principle reproduce the first-step OD/propagation/targeting chain, but that requires long custom Python and C/C++ module work rather than native mission-sequence or OD resources.
**NASA 42** fits the same second-step GNC and physics validation role when a maneuver product is imported into a dynamics simulation. Importing that product does not make it a mission-operations tool: it still lacks ground OD desks, pass scheduling, tracking-data ingestion workflows, and routine ops product generation.

# 6 Software architecture

## 6.1 Plugin / extension architecture

**1. FreeFlyer: The "Extensions SDK" (.NET/C#). FreeFlyer** has a highly formalized, enterprise-grade extension architecture known as the **FreeFlyer Extensions SDK**.

- **How it works:** You can write custom objects using C# or other .NET languages. For example, if you invent a brand-new solar radiation pressure model or a custom thruster logic, you write it in C#, compile it into a dynamic-link library (`.DLL`), and register it using the FreeFlyer Extensions Manager.
- **Capabilities:** Once registered, your custom code appears directly inside the FreeFlyer GUI alongside the native objects. You can create **Custom Force Extensions** (to add new physics) and **Custom Estimates Extensions** (to add new state variables into FreeFlyer's native Orbit Determination filter).

**2. GMAT: The "C++ Plugin Architecture". GMAT** is built with a native plug-in architecture designed to allow developers to inject new code without touching the massive core GMAT repository.

- **How it works:** GMAT plugins are typically written in C++. If you want to add a proprietary atmospheric drag model or a custom hardware interface, you write the C++ code, compile it into a shared library (a `.dll` on Windows or `.dylib`/`.so` on Mac/Linux), and drop it into GMAT's `plugins` folder.

- **Capabilities:** When GMAT boots up, it reads a startup file, detects the plugin, and dynamically loads your custom mathematical models, hardware interfaces, or optimizers directly into the GMAT script engine.

**3. Basilisk: Inherently Modular (C/C++/Python). Basilisk** doesn't just "support" plugins; its entire architecture is essentially a collection of plugins communicating over a message bus.

- **How it works:** Basilisk does not have a monolithic core. It is divided into an event-driven task manager and a message-passing system. To extend it, you simply write a **Custom Module** in C, C++, or Python.
- **Capabilities:** You can write a custom C++ module for a new star tracker algorithm, compile it, wrap it in Python using SWIG, and instantly drop it into your simulation python script. Because the architecture forces strict modular decoupling, your custom module can pass data to Basilisk's native modules seamlessly without needing a formal "Extensions Manager."

**4. Orekit: Programmatic Inheritance (Java/Python). Orekit does not have a plugin architecture**, because it is not a standalone host application. It is a low-level library.

- **How it works:** You do not build a plugin *for* Orekit; you build your application *around* Orekit.
- **Capabilities:** If you want to add a custom force model or a new coordinate frame, you simply write a new Java or Python class that inherits from Orekit's `ForceModel` or `Frame` interfaces. You extend the software programmatically using standard object-oriented programming principles.

**5. NASA 42: Source-Code Modification (C). NASA 42 does not support dynamic plugins or extensions**.

- **How it works:** 42 is heavily statically compiled C code. While it has IPC (Inter-Process Communication) sockets to talk to other software, if you want to add a new fundamental physical force or a new sensor model natively into 42, you must write the C code directly into the source files and recompile the entire software executable from scratch.

# 7 Summary table

| Capability | GMAT | Orekit | Basilisk | FreeFlyer | NASA 42 |
|---|---|---|---|---|---|
| *1. General* | | | | | |
| Open source | Yes | Yes | Yes | Commercial | Yes |
| GUI | Strong | — | — | Strong | OpenGL viewer |
| Script / API | GMAT Script + Python API | Java / Python Library only | Python + C++ modules | FreeForm / Runtime API | Input decks |
| MATLAB | Official API | Custom glue | Custom glue | Official API | Custom glue |
| *2. Core astrodynamics* | | | | | |
| Propagation | Strong | Strong | Strong; fewer perturbations | Strong; 360×360 MSIS | Dynamics-focused |
| Targeting | Strong (`Target`) | Strong | Custom script loop | Strong (`Target`) | — |
| Constellation | Formations; manual patterns | Good; custom viz | Walker sim; custom coverage | Strongest design tool | Many SC; not design |
| Interplanetary | Strong design/refinement | Strong; CR3BP | Flyby sim + GNC; not design | Strong design/refinement | Dynamics; not design |
| *3. Advanced analysis* | | | | | |
| Optimization | Strong | Custom; ext. optimizer | Test trajectory; ext. Python | Strong | — |
| OD (ground) | Strong ready | Strong | Onboard/opnav only | Strong ready | — |
| Monte Carlo | Scripted / API | User loops; Taylor fields | Ready campaigns | Ready campaigns | External scripts |
| Conjunction | Prop/OD engine; scripted Pc | Strong; SSA/CDM | Relative motion; ext. Pc | Strong ready Pc | — |
| Coverage | Contact, eclipse | Strong; custom reports | Point imaging; custom grids | Richest ready tool | Sensors only |
| *4. GNC / simulation* | | | | | |
| Dynamics & GNC | Attitude models only | Attitude models only | Strong closed-loop | Attitude states; no RW ADCS | Strong closed-loop |
| FSW development | — | — | Strongest MIL/SIL | — | `PROTOTYPE_FSW`, `42fsw.c`; NOS3→cFS |
| HIL simulation | — | — | Custom HIL; not turnkey | — | NOS3 HIL stack; 42 alone RT only |
| Visualization | Semi-live 3D + plots | Offline export/replay | Live 3D (Vizard) | Semi-live 3D + plots | Live OpenGL 3D |
| *5. Mission operations* | | | | | |
| Mission ops | Native ground FD | Custom stack | Ops-adjacent (step 2) | Native ground FD | GNC validation only |
| *6. Software architecture* | | | | | |
| Extensions | C++ plugins | Programmatic (Java) | Modular message bus | Extensions SDK (.NET) | Source rebuild |

**Selection guide.** Use **GMAT** or **FreeFlyer** for ground flight-dynamics operations products, targeting, finite burns, ground OD, and analyst visualization. **FreeFlyer** is the strongest ready desktop choice for constellation design, coverage, conjunction $P_c$, and Monte Carlo campaigns. Use **Orekit** as the propagation, estimation, and access engine inside custom CATEC software stacks. Use **Basilisk** for spacecraft dynamics, closed-loop GNC, MIL/SIL FSW development, custom HIL benches, and live 3D review; use it as a second-step validator after GMAT/FreeFlyer produce a maneuver or ephemeris product. Use **NASA 42** for high-fidelity 6-DOF dynamics, flexible-body studies, and closed-loop ADCS/GNC with the same input decks used in stand-alone runs. Pair with **NOS3** when the project needs cFS flight software and a synchronized HIL bench rather than Basilisk-style Python MIL/SIL. NASA 42 is not a mission-operations desk or trajectory-design tool.

# 8 Recommended tools by capability

Tools are listed **left to right, best fit first**. A comma separates different ranks; a hyphen separates tools **tied at the same rank** (alphabetical within a tie). The final choice still depends on team experience, project pipeline, license budget, and whether a desktop workflow or embedded library is preferred.

| **Capability** | **Recommended tool(s)** |
|---|---|
| *1. General* | |
| Open source | Basilisk - GMAT - NASA 42 - Orekit |
| Desktop GUI | FreeFlyer - GMAT, NASA 42 |
| Script and API | Orekit, Basilisk, GMAT, FreeFlyer, NASA 42 |
| MATLAB integration | FreeFlyer - GMAT, Orekit |
| *2. Core astrodynamics* | |
| Orbit propagation | Orekit, FreeFlyer, GMAT, Basilisk, NASA 42 |
| Maneuver targeting | FreeFlyer - GMAT, Orekit, Basilisk |
| Constellation | FreeFlyer, GMAT, Orekit, Basilisk, NASA 42 |
| Interplanetary missions | FreeFlyer - GMAT, Orekit, Basilisk, NASA 42 |
| *3. Advanced mission analysis* | |
| Trajectory optimization | FreeFlyer - GMAT, Orekit, Basilisk |
| Orbit determination | FreeFlyer - GMAT, Orekit |
| Monte Carlo analysis | Basilisk - FreeFlyer, GMAT, Orekit, NASA 42 |
| Conjunction analysis | FreeFlyer, Orekit, GMAT, Basilisk |
| Coverage analysis | FreeFlyer, GMAT, Orekit, Basilisk, NASA 42 |
| *4. GNC / simulation* | |
| Spacecraft dynamics and GNC | Basilisk, NASA 42, GMAT - Orekit - FreeFlyer |
| Flight software development | Basilisk - NASA 42, NOS3 (cFS) |
| Real-time / HIL simulation | Basilisk, NOS3 - NASA 42 |
| Live visualization | Basilisk, NASA 42, FreeFlyer - GMAT |
| *5. Mission operations* | |
| Mission operations | FreeFlyer - GMAT, Orekit, Basilisk - NASA 42 |
| *6. Software architecture* | |
| Plugin architecture | Orekit, Basilisk, FreeFlyer - GMAT |

# 9 Selection guide

No single tool covers a full CubeSat lifecycle. The practical split is by workflow, not by which tool has the longest feature list.

**Ground flight dynamics and mission analysis.** For orbit propagation, maneuver targeting, ground orbit determination, constellation design, coverage, conjunction analysis, trajectory optimization, and routine post-launch operations products, start with **FreeFlyer** or **GMAT**. **FreeFlyer** is the stronger ready desktop choice when constellation layout, rich coverage products, conjunction $P_c$, and Monte Carlo campaigns are central. **GMAT** is equally credible for navigation-style workflows, scripting, and high-fidelity perturbation studies. Choose **Orekit** when CATEC is building custom Java/Python mission or operations software and needs a propagation, estimation, and access engine rather than an analyst console.

**Spacecraft simulation, FSW, and ADCS validation.** Use **Basilisk** for closed-loop dynamics and GNC, reusable C/C++ flight- software modules, custom real-time/HIL benches, and live 3D review. After **FreeFlyer** or **GMAT** produces a maneuver or ephemeris product, **Basilisk** is the natural second step to verify reaction-wheel slews, actuator limits, sun constraints, and onboard algorithm behavior before uplink.

**High-fidelity multibody dynamics and cFS FSW.** Use **NASA 42** for 6-DOF dynamics, flexible-body effects, and closed-loop ADCS/GNC with `PROTOTYPE_FSW`, custom `42fsw.c`, or ACS IPC to external code. Use **NOS3** on top of the same 42 decks when the deliverable is cross-compiled **cFS** applications, simulated avionics, ground commanding, and a synchronized HIL bench. Standalone 42 is not a turnkey HIL platform; NOS3 supplies that layer. Neither is a mission-operations desk, trajectory-design tool, or ground OD platform.

**Typical CATEC combinations.** A common pipeline is **FreeFlyer/GMAT** for flight-dynamics and operations analysis, **Basilisk** for fast Python-orchestrated MIL/SIL and custom HIL glue, and **NASA 42 + NOS3** when the project already targets cFS and needs the same physics deck from GNC prototyping through FSW-in-the-loop. **Orekit** fits when embedded mission software is part of the project.